

AIAA 2205: Introduction to AI

Text to Speech Synthesis (TTS)

Li LIU

avrillliu@hkust-gz.edu.cn

The Hong Kong University of Science and Technology (Guangzhou)

Outline

- Introduction to Text to Speech (TTS)
- WaveNet
- Tacotron
- F5-TTS

Outline

- Introduction to Text to Speech (TTS)
- WaveNet
- Tacotron
- F5-TTS

Text to Speech (TTS)



Text to speech (TTS) is a technology that converts text into spoken audio.



 **Speech**

Body gesture

Basic Pipeline of TTS

1. Text Preprocessing:

- Converts input text into a format suitable for the model.
- Tasks include text normalization (*e.g.*, expanding abbreviations, converting numbers to words), grapheme-to-phoneme conversion, and prosody prediction.

2. Acoustic Model:

- Generates acoustic features (*e.g.*, mel-spectrograms) from the processed text.
- This is the core component that determines the quality and naturalness of the synthesized speech.

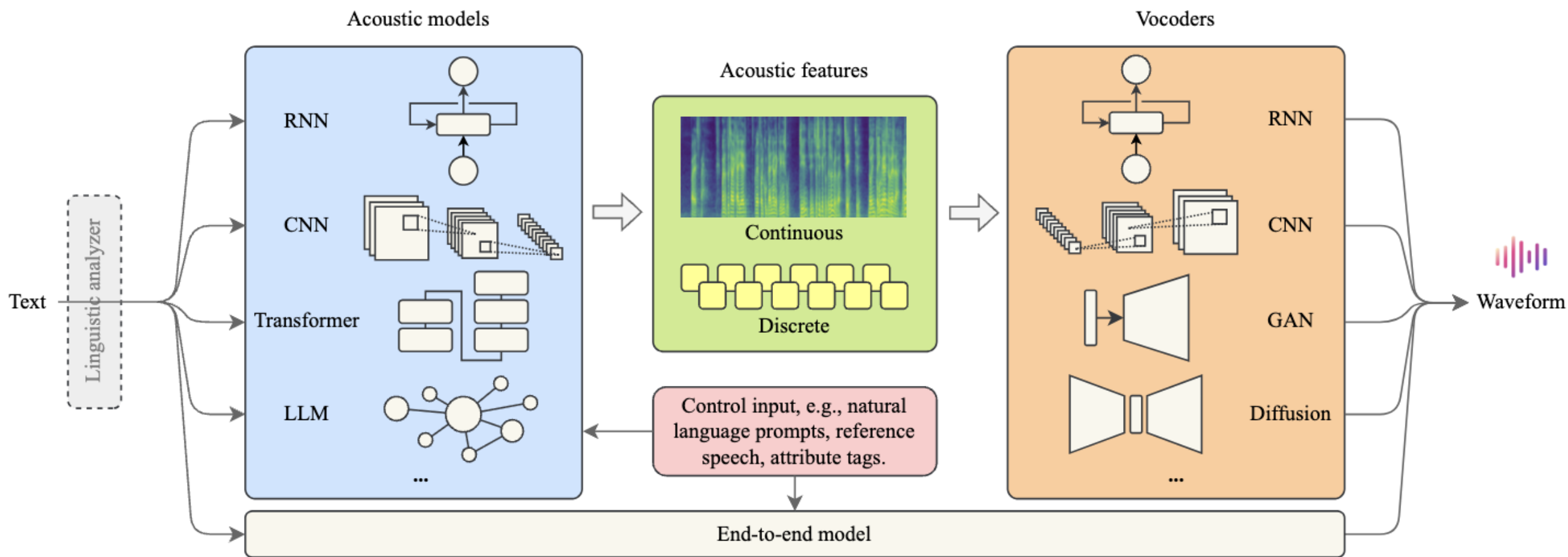
3. Vocoder:

- Converts the acoustic features (*e.g.*, mel-spectrograms) into a waveform signal.
- Modern vocoders use neural networks to generate high-fidelity speech.

4. Post-Processing:

- Enhances the generated speech by applying techniques such as noise reduction, pitch adjustment, or volume normalization.

General pipeline of controllable TTS



Traditional Methods

Concatenative approaches vs. Statistical Parametric approach

- Concatenative: More natural sounding, Less flexible, Take up more space
- Statistical Parametric: More error, more flexible, Smaller models

Traditionally, two stages: frontend and backend

- Frontend analyzes text and determines phonemes, stresses, pitch, etc.
- Backend generates the audio

Moving towards models which can convert directly from text to audio, with the model itself learning any intermediate representations necessary.

Outline

- Introduction to Text to Speech (TTS)
- **WaveNet**
- Tacotron
- F5-TTS

WaveNet: A Generative Model for Raw Audio (2016), Google

Operates on **Raw Waveform** and generates a raw waveform (audio samples)

Each audio sample's predictive distribution conditioned on all previous ones

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1})$$

Problems to overcome:

- Tons of samples: 16kHz = 16,000 samples per second
- Correlations in both small time scales and large time scales
- 16 bit audio means 65,536 probabilities per time step



Figure 1: A second of generated speech.

Problem: Too many samples, and different time scales of correlation

Solution: Causal Convolutions

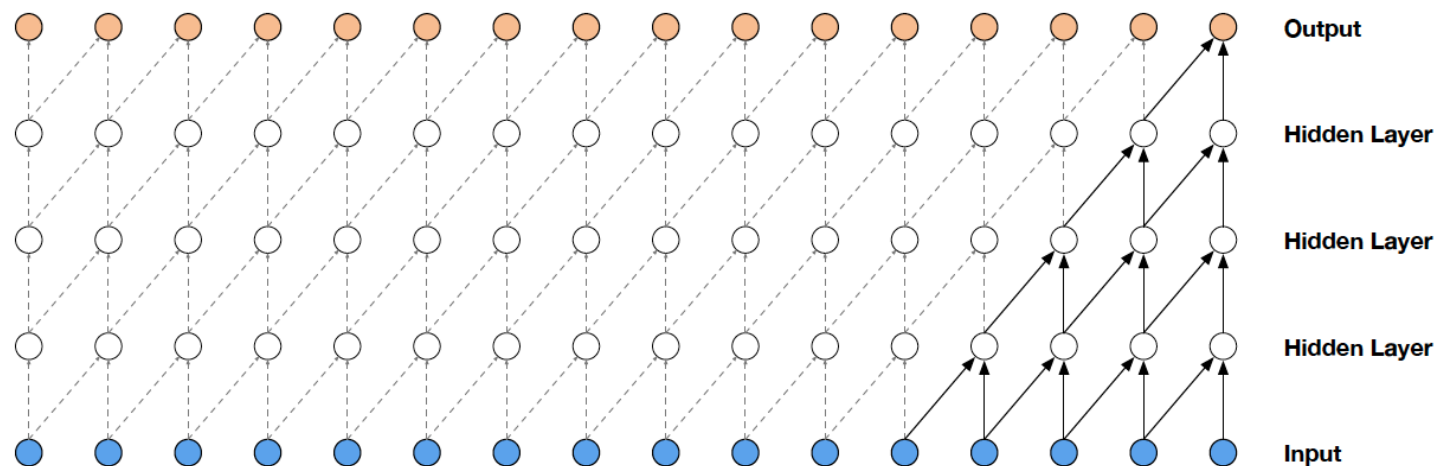


Figure 2: Visualization of a stack of causal convolutional layers.

The main ingredient of WaveNet are causal convolutions. By using causal convolutions, we make sure the model cannot violate the ordering in which we model the data: the prediction $p(x_{t+1} \mid x_1, \dots, x_t)$ emitted by the model at timestep t cannot depend on any of the future timesteps.

Causal Convolutions

Prediction of an audio sample can only depend on timestamps before it (nothing from the future)

No recurrent connections (Not an RNN) => Faster to train than RNN's

- Especially with very long sequences

Key problem: Requires many layers to increase the receptive field

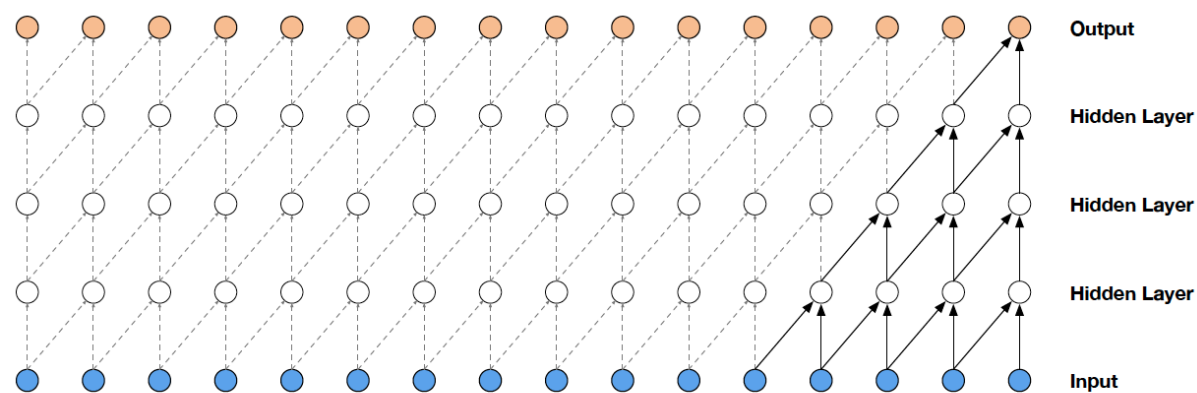


Figure 2: Visualization of a stack of causal convolutional layers.

Dilated Causal Convolutions

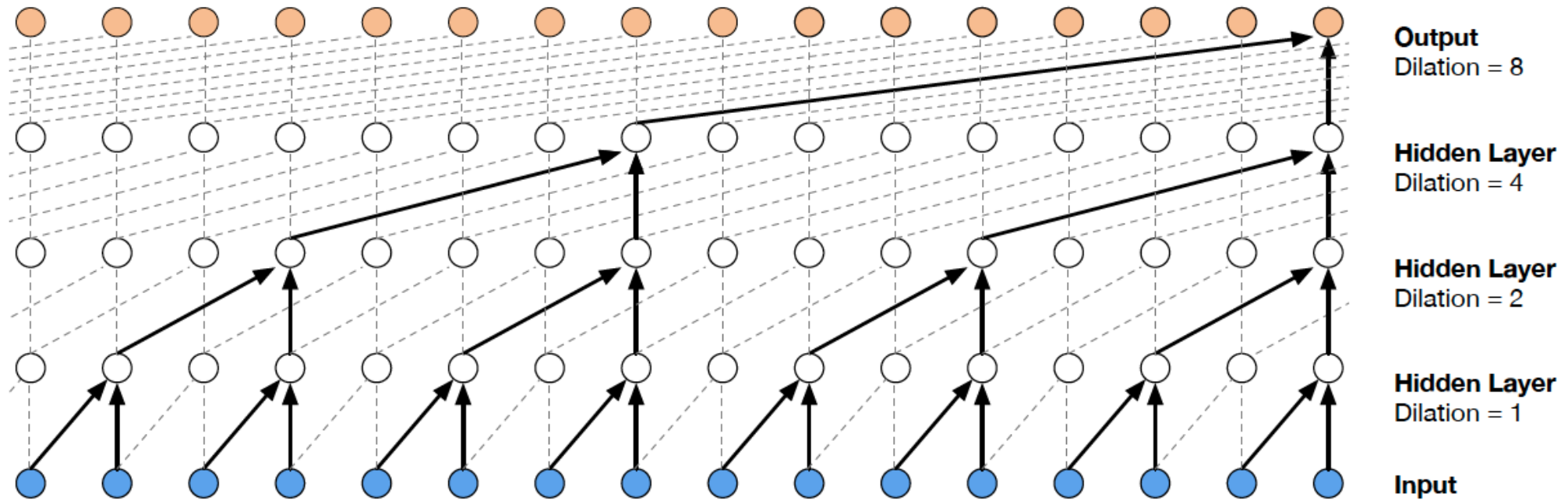


Figure 3: Visualization of a stack of *dilated* causal convolutional layers.

Dilated Causal Convolutions

Filter is applied to area much larger than its length by skipping inputs at each layer: Large receptive field

Allows information to be learned from both temporally close and far samples

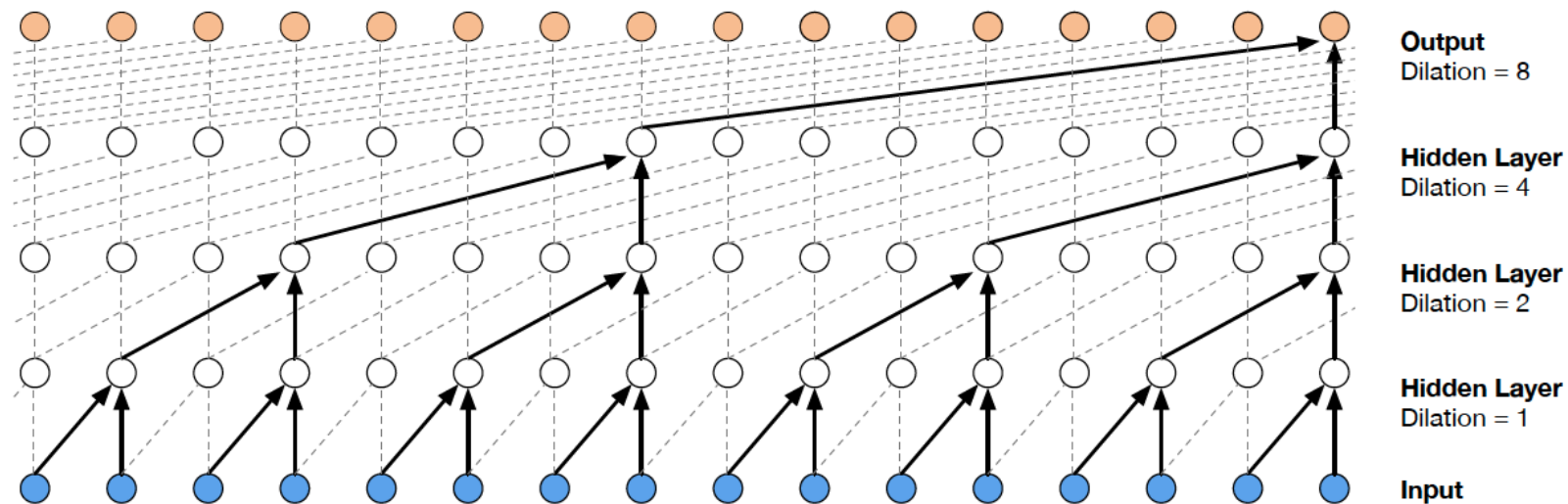


Figure 3: Visualization of a stack of *dilated* causal convolutional layers.

Dilated Causal Convolutions

- The intuition behind dilated convolutions configuration is two-fold.
 - ✓ First, exponentially increasing the dilation factor results in exponential receptive field growth with depth (Yu & Koltun, 2016).
 - ✓ Second, stacking these blocks further increases the model capacity and the receptive field size.

- ✓ Skip connections are used throughout the network
- ✓ Speeds up convergence
- ✓ Enables training of deeper models

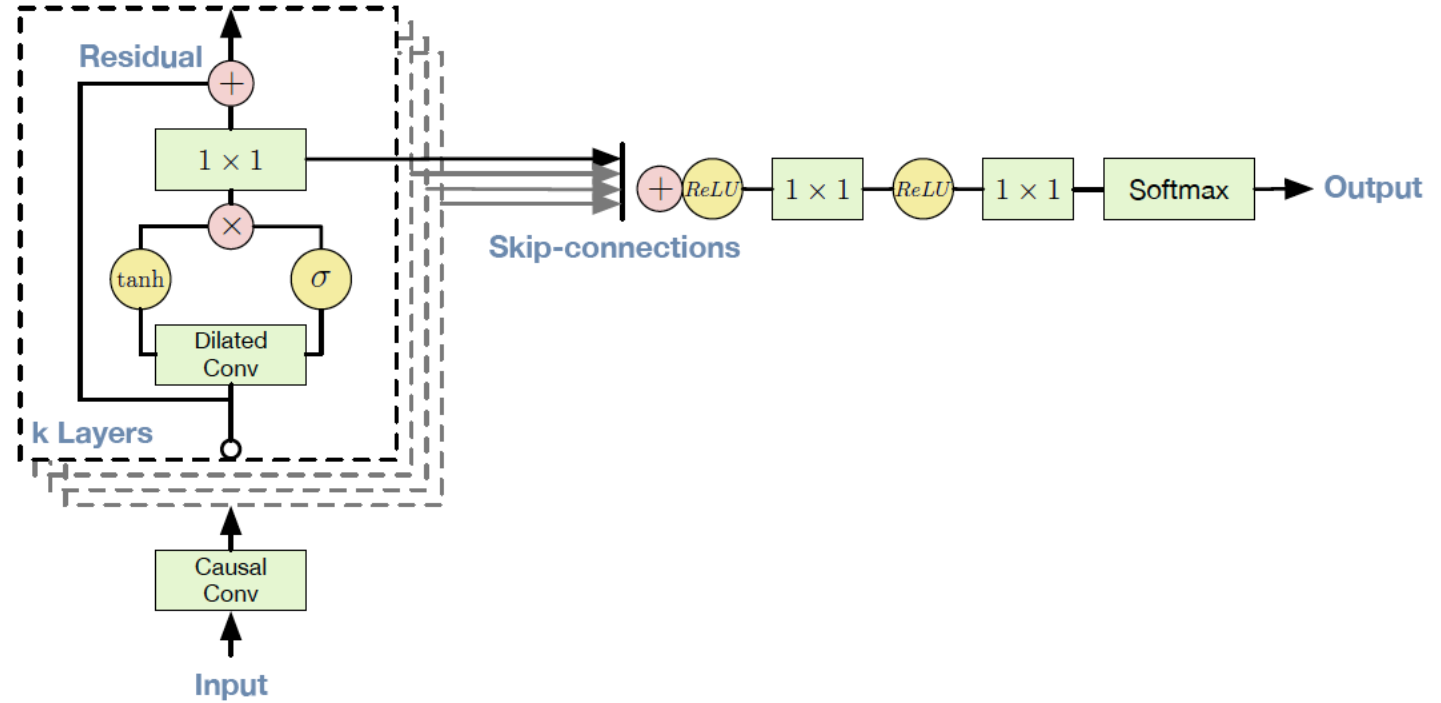


Figure 4: Overview of the residual block and the entire architecture.

Add in inputs

Generic enough to allow for both universal and local input

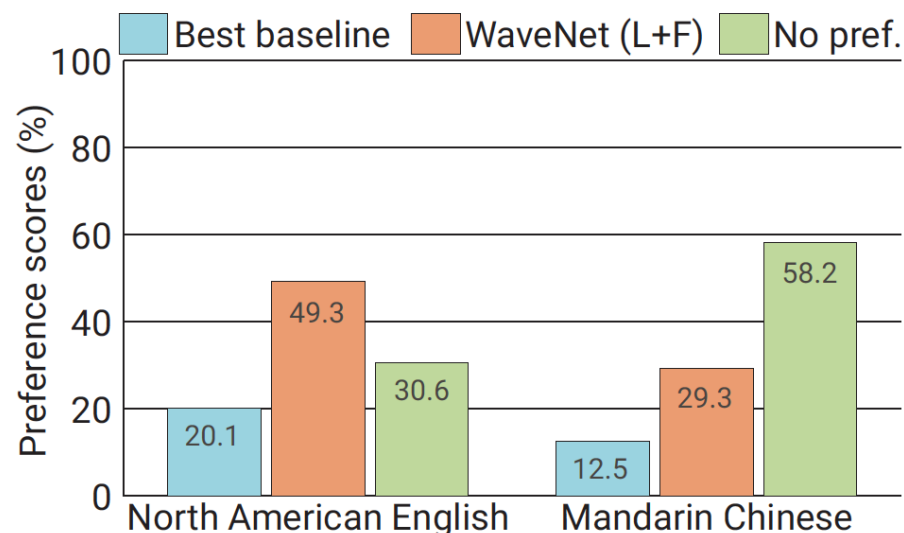
- Universal: Speaker Identity
- Local: Phonemes, inflection, stress

$$p(\mathbf{x} \mid \mathbf{h}) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1}, \mathbf{h})$$

Results

Speech samples	Subjective 5-scale MOS in naturalness	
	North American English	Mandarin Chinese
LSTM-RNN parametric	3.67 ± 0.098	3.79 ± 0.084
HMM-driven concatenative	3.86 ± 0.137	3.47 ± 0.108
WaveNet (L+F)	4.21 ± 0.081	4.08 ± 0.085
Natural (8-bit μ -law)	4.46 ± 0.067	4.25 ± 0.082
Natural (16-bit linear PCM)	4.55 ± 0.075	4.21 ± 0.071

30.6% of people think that both are equally good and it is hard to tell which one is better (No preference).



Outline

- Introduction to Text to Speech (TTS)
- WaveNet
- **Tacotron**
- F5-TTS

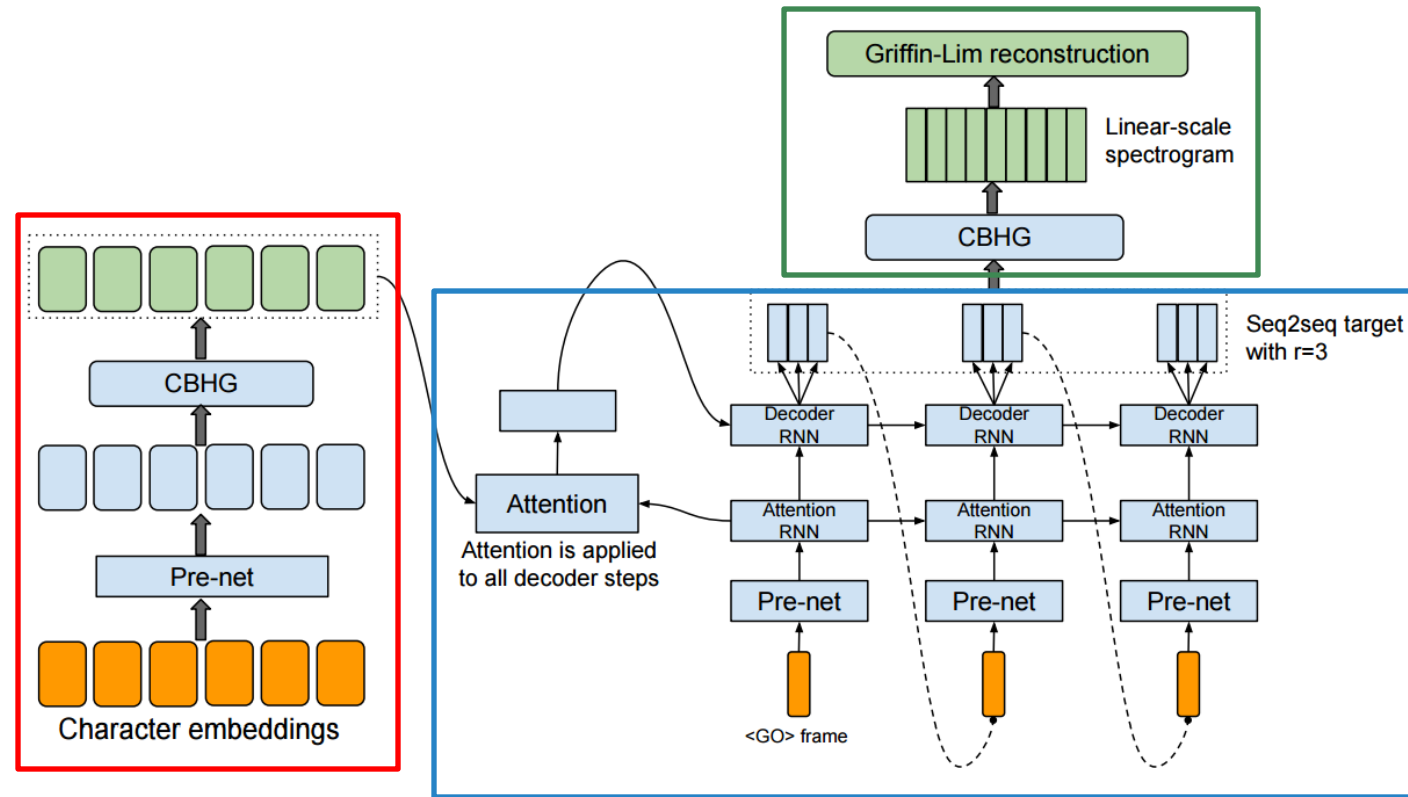
Tacotron: Towards End-to-End Speech Synthesis

Truly end-to-end TTS pipeline by Google

- Reduces feature engineering
- Allows conditioning on various attributes

Architecture

- One network based on the sequence-to-sequence with attention paradigm
- Red: Encoder
- Blue: Decoder
- Green: Post-processing net



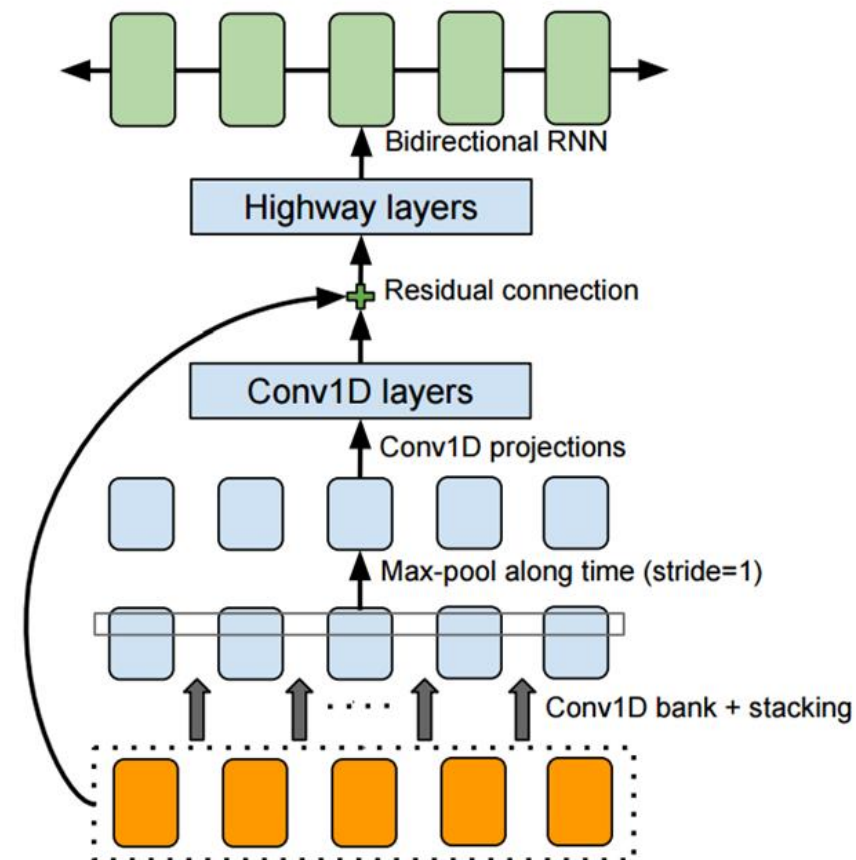
Tacotron CBHG Module

1D Convolutional Bank + highway network +
bidirectional GRU (CBHG)

- Module for extracting representations from sequences
- Inspired by work from “Fully Character-Level Neural Machine Translation without Explicit Segmentation”

Architecture

- Bank of 1D convolutional filters
- Highway networks
 - Gating unit learn to regulate flow of information through network
- Bidirectional GRU RNN



Highway network

A Highway Network uses **learned gating mechanisms** to control how much of the input is passed through directly to the output and how much is transformed:

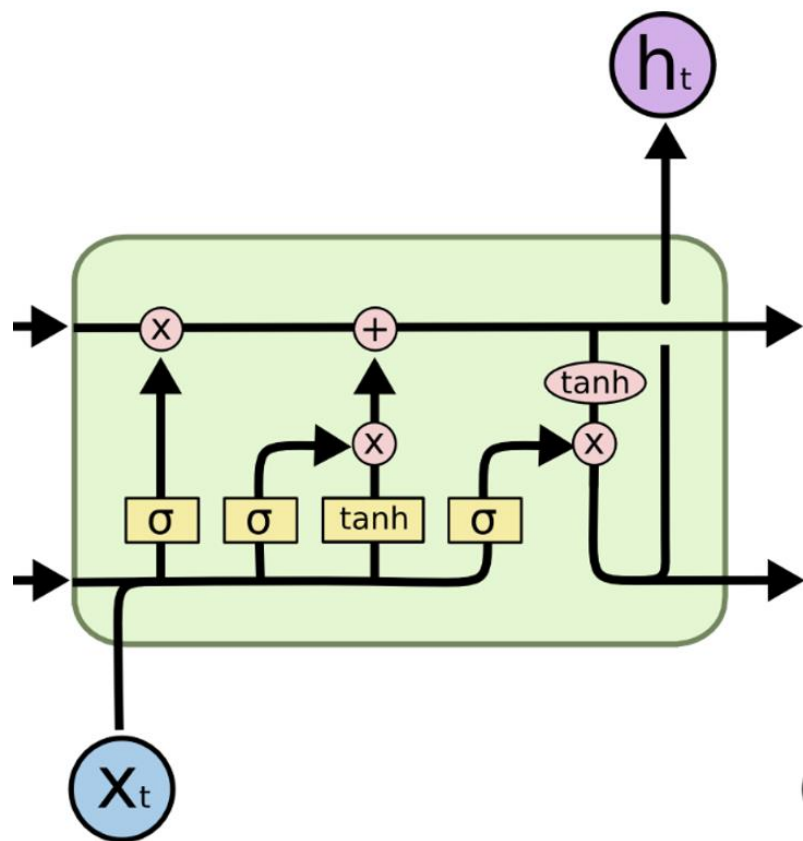
$$\mathbf{y} = \mathbf{H}(\mathbf{x}, \mathbf{W}_H) \cdot \mathbf{T}(\mathbf{x}, \mathbf{W}_T) + \mathbf{x} \cdot (1 - \mathbf{T}(\mathbf{x}, \mathbf{W}_T))$$

Where:

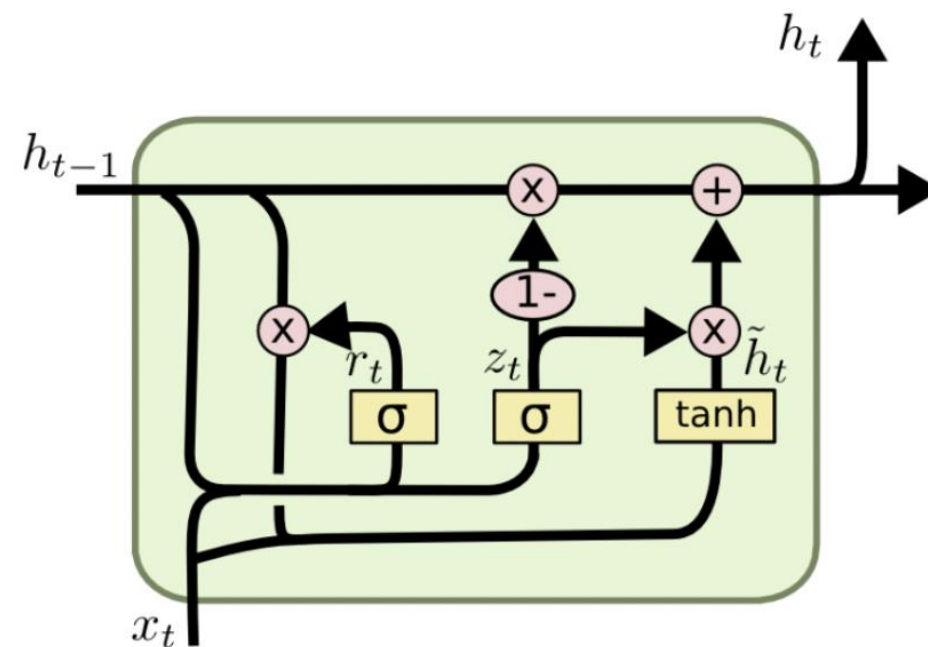
- $\mathbf{x}$: Input
- $\mathbf{H}(\cdot)$: A non-linear transformation (like a regular feedforward layer)
- $\mathbf{T}(\cdot)$: The **transform gate** (typically a sigmoid)
- $(1 - \mathbf{T}(\cdot))$: The **carry gate**, which controls how much of the input should be carried through unchanged
- If $\mathbf{T}$ outputs values near 1 $\rightarrow$ the network learns to **transform** the input
- If $\mathbf{T}$ outputs values near 0 $\rightarrow$ the network **preserves** the input

GRU (Gated Recurrent Neural Networks)

LSTM:



GRU:



1. Update gate z_t

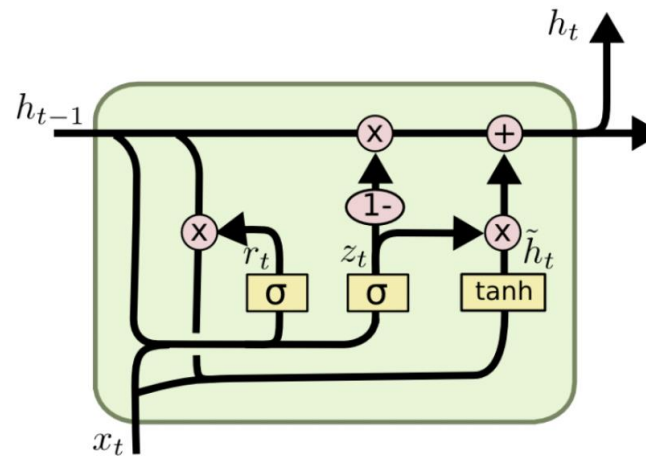
- Controls how much of the previous hidden state h_{t-1} is carried forward.
- Formula:

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$

2. Reset gate r_t

- Controls how much of the past information to forget when computing the candidate state.
- Formula:

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$



3. Candidate hidden state $\tilde{h}_t$

- A temporary state proposal, computed using:

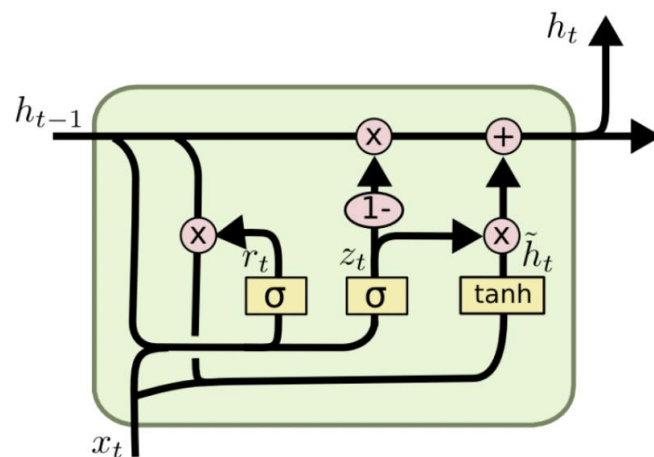
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$$

- $r_t \odot h_{t-1}$: element-wise multiplication (shown in the diagram with a pink $\times$)

4. Final hidden state h_t

- This is a linear interpolation between the previous hidden state h_{t-1} and the candidate $\tilde{h}_t$, controlled by z_t :

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

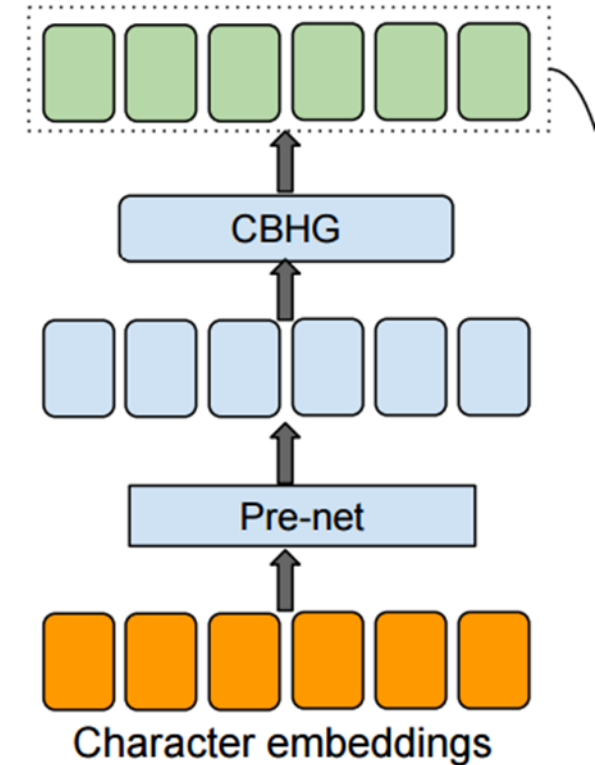


Tacotron Encoder

Extracts robust sequential representation of text

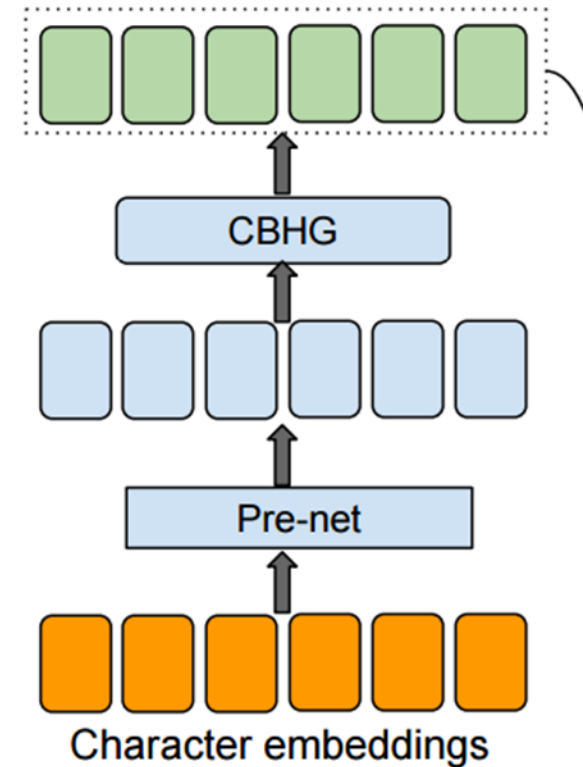
Architecture

- Input: one-hot vector of characters embedded into a continuous sequence
- Pre-Net, a set of on non-linear transformations
- CBHG module



Tacotron Encoder

- Pre-Net, a set of non-linear transformations
- It typically consists of one or more fully connected layers with a non-linear activation function (such as ReLU or sigmoid) applied to each layer.
- Usage: Dimensionality reduction; Non-linear transformation and Regularization



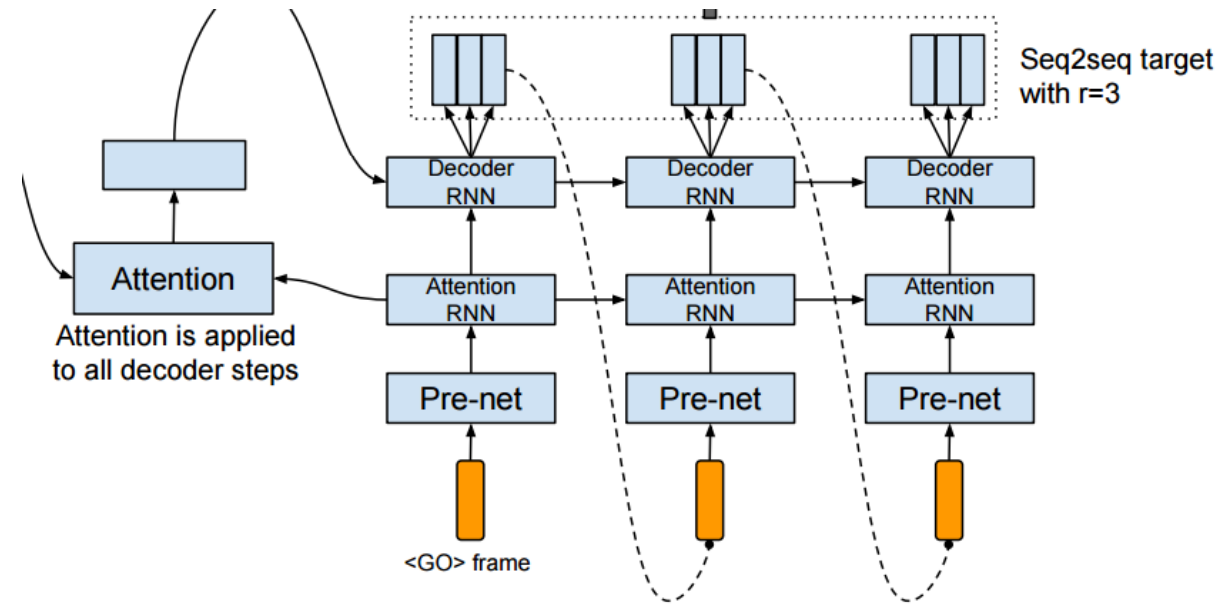
Tacotron Decoder

Target is 80-band mel-scale spectrogram rather than raw waveform

- Waveform is highly redundant representation

Architecture

- Attention RNN: 1 layer 256 GRU cells
- Decoder RNN: 2 layer residual 256 GRU cells
- Fully connected output layer

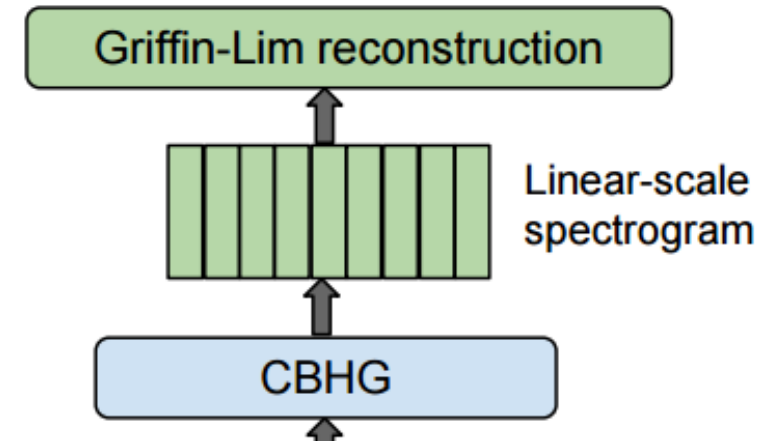


Tacotron Post-Processing Network

Convert sequence-to-sequence target to waveform
though can be used to predict alternative targets

Architecture

- CBHG
- Griffin-Lim algorithm
 - Estimates waveform from spectrogram
 - Uses an iterative algorithm to decrease MSE
 - Used for simplicity



Griffin-Lim algorithm

- Many speech processing algorithms, those of speech enhancement in particular, modify or estimate the magnitude spectrum, such that the phase spectrum is either very noisy or completely unavailable.
- Yet it is desirable to recover an estimate of the complex spectrum. The method known as the *Griffin-Lim* **algorithm**.

Outline

- Introduction to Text to Speech (TTS)
- WaveNet
- Tacotron
- **F5-TTS**

F5-TTS

Chen, Yushen, Zhikang Niu, Ziyang Ma, Keqi Deng, Chunhui Wang, Jian Zhao, Kai Yu, and Xie Chen. "F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching." arXiv preprint arXiv:2410.06885 (2024).

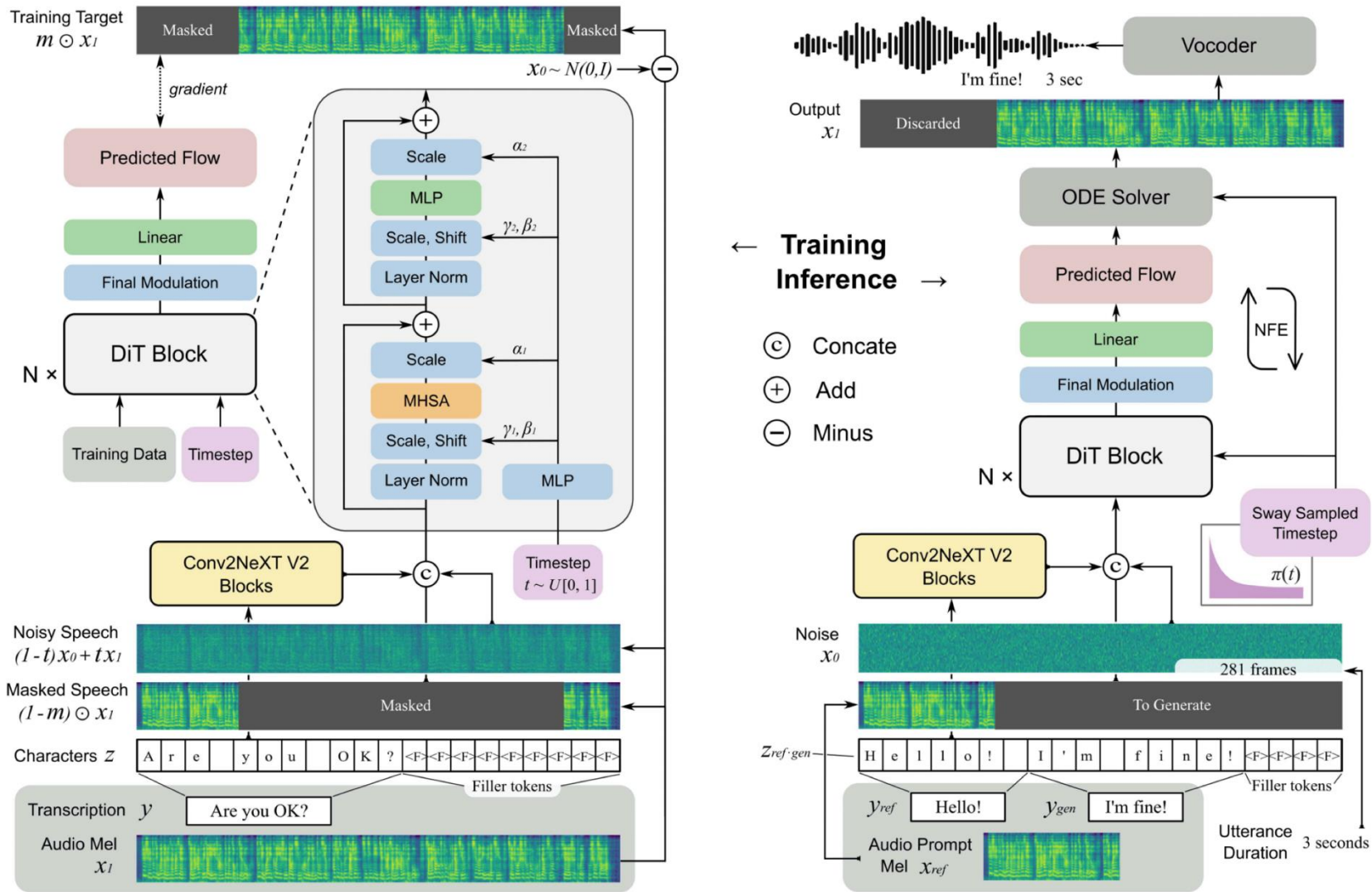


Figure 1: An overview of F5-TTS training (left) and inference (right). The model is trained on the text-guided **speech-infilling task** and **condition flow matching** loss. The input text is converted to a character sequence, padded with filler tokens to the same length as input speech, and refined by **ConvNeXt** blocks before concatenation with speech input. The inference leverages **Sway Sampling** for flow steps, with the model and an **ODE solver** to generate speech from sampled noise.

F5-TTS

- Diffusion Model
- Flow Matching
 - <https://zhuanlan.zhihu.com/p/685921518>

Thank you!